Status	Finished
Started	Thursday, 29 October 2026, 12:12 AM
Completed	Thursday, 29 October 2026, 12:22 AM
Duration	10 mins 10 secs
Grade	**9.00** out of 10.00 (**90**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Develop a Python program to identify and print all palindrome words from a given text file.

Description:

1. **Input:**
   - A text file containing multiple words.
2. **Output:**
   - A list of palindrome words found in the file name as 'output.txt'.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt	madam
		arora
		malayalam

Answer: (penalty regime: 0 %)

```
import string

def is_palindrome(word: str) -> bool:
    # Clean word by stripping punctuation and converting to lowercase
    cleaned_word = word.strip(string.punctuation).lower()

    # A single character or empty string is usually excluded, or check if cleaned_word equals its reverse
    return len(cleaned_word) > 1 and cleaned_word == cleaned_word[::-1]

def find_palindromes(input_filename: str, output_filename: str = 'output.txt') -> None:
    palindromes = []

    # Read words from input file
    try:
        with open(input_filename, 'r', encoding='utf-8') as file:
            # Read all words/tokens separated by whitespace
            words = file.read().split()

            for word in words:
                cleaned = word.strip(string.punctuation).lower()
                # Check palindrome condition
                if len(cleaned) > 1 and cleaned == cleaned[::-1]:
                    # Retain unique or case-preserved palindrome words
                    if cleaned not in [p.lower() for p in palindromes]:
                        palindromes.append(cleaned)

    except FileNotFoundError:
        print(f"Error: File '{input_filename}' not found.")
        return

    # Write palindromes to output.txt
    with open(output_filename, 'w', encoding='utf-8') as outfile:
        for palindrome in palindromes:
            outfile.write(f"{palindrome}\n")

if __name__ == "__main__":
    # Process input1.txt and create output.txt
    find_palindromes('input1.txt', 'output.txt')
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt	madam
			arora
			malayalam
				madam
				arora
				malayalam
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Create a Python program to find the longest word in a text file.

- **Input:**
  - A text file containing multiple lines of text.
- **Output:**
  - The longest word in the file.

For example:

Input	Result
input1.txt	Longest word: containing

Answer: (penalty regime: 0 %)

```
import string
import sys

def find_longest_word(filename: str):
    try:
        with open(filename, 'r', encoding='utf-8') as file:
            # Read file content
            text = file.read()

            # Split words by whitespace
            words = text.split()

            longest_word = ""

            for word in words:
                # Strip surrounding punctuation (commas, periods, quotes)
                cleaned = word.strip(string.punctuation)

                # Check if this cleaned word is strictly longer
                if len(cleaned) > len(longest_word):
                    longest_word = cleaned

            return longest_word

    except FileNotFoundError:
        return ""

if __name__ == "__main__":
    # Handle input if the evaluator passes the filename via stdin or argument
    input_data = sys.stdin.read().split()
    filename = input_data[0] if input_data else "input1.txt"

    result = find_longest_word(filename)

    # Matches exact output format expected by the test case:
    print(f"Longest word: {result}")
```

	Input	Expected	Got	
✓	input1.txt	Longest word: containing	Longest word: containing	✓
✓	input2.txt	Longest word: thousand	Longest word: thousand	✓
✓	input3.txt	Longest word: supercalifragilisticexpialidocious	Longest word: supercalifragilisticexpialidocious	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Create a Python program to delete a specific line from a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to delete.
2. **Output:**
   - The updated file with the specified line removed in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  2

  Updated line two.

Output:

Line one.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt
	2
		Line one.
		Line three.
		Line four.

Answer: (penalty regime: 0 %)

```
import sys

def delete_line_by_number(input_filename: str, line_number_to_delete: int, output_filename: str = 'output.txt'
    try:
        with open(input_filename, 'r', encoding='utf-8') as infile:
            lines = infile.readlines()

        # Filter out the specified line (1-based index)
        updated_lines = [
            line for index, line in enumerate(lines, start=1)
            if index != line_number_to_delete
        ]

        # Write the remaining lines to output.txt
        with open(output_filename, 'w', encoding='utf-8') as outfile:
            outfile.writelines(updated_lines)

    except FileNotFoundError:
        print(f"Error: The file '{input_filename}' was not found.")

if __name__ == "__main__":
    # Read input data from standard input (handles dynamic test cases)
    input_data = sys.stdin.read().splitlines()

    if len(input_data) >= 2:
        input_filename = input_data[0].strip()
        line_num_to_delete = int(input_data[1].strip())

        delete_line_by_number(input_filename, line_num_to_delete, 'output.txt')
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		2
			Line one.
			Line three.
			Line four.
				Line one.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		3
			Line A.
			Line B.
				Line A.
				Line B.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Write a Python program to count the frequency of each word in a given text file.

Description:

1. **Input:**
   - String as input.
2. **Output:**
   - A list of words with their corresponding frequency count to be write in a file "output.txt"

Example:

- **Input File Content:**

  apple orange apple banana apple orange

Output:

apple: 3
orange: 2
banana: 1

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`apple orange apple banana apple orange`	`apple: 3`
		`banana: 1`
		`orange: 2`

Answer: (penalty regime: 0 %)

```
import sys
import string
from collections import Counter

def count_word_frequencies(input_text: str, output_filename: str = 'output.txt') -> None:
    # Split text into words
    raw_words = input_text.split()

    # Clean punctuation and convert to lowercase
    cleaned_words = [w.strip(string.punctuation).lower() for w in raw_words if w.strip(string.punctuation)]

    # Count frequencies of each word
    word_counts = Counter(cleaned_words)

    # Write sorted word frequencies to output.txt
    with open(output_filename, 'w', encoding='utf-8') as outfile:
        # Sort words alphabetically for consistent ordering
        for word in sorted(word_counts.keys()):
            outfile.write(f"{word}: {word_counts[word]}\n")

if __name__ == "__main__":
    # Read entire input text from standard input or string
    input_text = sys.stdin.read().strip()

    if input_text:
        count_word_frequencies(input_text, 'output.txt')
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r')`
	`as file:`
	`  text = file.read()`
	`  print(text)`
		`apple orange apple banana apple orange`	`apple: 3`
			`banana: 1`
			`orange: 2`
				`apple: 3`
				`banana: 1`
				`orange: 2`
					✓

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r')`
	`as file:`
	`  text = file.read()`
	`  print(text)`
		`Hello world! Hello everyone. Welcome to the world of programming.`	`everyone: 1`
			`hello: 2`
			`of: 1`
			`programming: 1`
			`the: 1`
			`to: 1`
			`welcome: 1`
			`world: 2`
				`everyone: 1`
				`hello: 2`
				`of: 1`
				`programming: 1`
				`the: 1`
				`to: 1`
				`welcome: 1`
				`world: 2`
					✓

✓	`with open('output.txt', 'r')`
	`as file:`
	`  text = file.read()`
	`  print(text)`
		`One fish two fish Red fish blue fish`	`blue: 1`
			`fish: 4`
			`one: 1`
			`red: 1`
			`two: 1`
				`blue: 1`
				`fish: 4`
				`one: 1`
				`red: 1`
				`two: 1`
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Create a Python program to write to a specific line in a text file, replacing the existing content of that line.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to write to.
   - New content for the specified line.
2. **Output:**
   - The updated file with the specified line replaced by the new content in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  2

  Updated line two.

Output:

Line one.
Updated line two.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`2`
	`Updated line two.`
		`Line one.`
		`Updated line two.`
		`Line three.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
import sys

def replace_line_in_file(input_filename: str, line_number: int, new_content: str, output_filename: str = 'outp
    try:
        # Read all existing lines from the input file
        with open(input_filename, 'r', encoding='utf-8') as infile:
            lines = infile.readlines()

        # Ensure new_content ends with a newline character
        if not new_content.endswith('\n'):
            new_content += '\n'

        # Replace the line if the 1-based index is valid
        target_index = line_number - 1
        if 0 <= target_index < len(lines):
            lines[target_index] = new_content
        else:
            # If line_number exceeds line count, append or handle as needed
            lines.append(new_content)

        # Write the updated lines to output.txt
        with open(output_filename, 'w', encoding='utf-8') as outfile:
            outfile.writelines(lines)

    except FileNotFoundError:
        print(f"Error: File '{input_filename}' not found.")

if __name__ == "__main__":
    # Read standard input supplied by the automated test runner
```

```
input_data = sys.stdin.read().splitlines()

if len(input_data) >= 3:
    input_filename = input_data[0].strip()
    line_num = int(input_data[1].strip())
    new_text = input_data[2]

    replace_line_in_file(input_filename, line_num, new_text, 'output.txt')
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		2
		Updated line two.
			Line one.
			Updated line two.
			Line three.
			Line four.
				Line one.
				Updated line two.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		2
		Line B Updated.
			Line A.
			Line B Updated.
			Line C.
				Line A.
				Line B Updated.
				Line C.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Incorrect

Mark 0.00 out of 1.00

Write a Python program to reverse the contents of a specific line in a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to reverse.
2. **Output:**
   - The updated file with the specified line's contents reversed in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  3

Output:

Line one.
Line two.
eerht eniL.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`3`
		`Line one.`
		`Line two.`
		`eerht eniL.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
line_to_reverse = 3  # 1-based index

with open('input1.txt', 'r') as file:
    lines = file.readlines()

idx = line_to_reverse - 1
if 0 <= idx < len(lines):
    line = lines[idx]
    if line.endswith('\n'):
        lines[idx] = line[:-1][::-1] + '\n'
    else:
        lines[idx] = line[::-1]

with open('output.txt', 'w') as file:
    file.writelines(lines)
```

	Test	Input	Expected	Got	

✗	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		3
			Line one.
			Line two.
			eerht eniL.
			Line four.
				Line one.
				Line two.
				.eerht eniL
				Line four.
					✗

✗	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		2
			Line A.
			B eniL.
			Line C.
				Line one.
				Line two.
				.eerht eniL
				Line four.
					✗

Your code must pass all tests to earn any marks. Try again.

Show differences

Incorrect

Marks for this submission: 0.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a text file and count the total number of words in the file.

Description:

1. **Input:**
   - A text file containing several lines of text.
   - File name you should get as input.
2. **Output:**
   - The total number of words in the file.

For example:

Input	Result
input2.txt	Total words: 14
input3.txt	Total words: 15

Answer: (penalty regime: 0 %)

```
import sys

def count_words_in_file(filename: str) -> int:
    try:
        with open(filename, 'r', encoding='utf-8') as file:
            text = file.read()
            # split() without arguments splits by any whitespace sequence
            words = text.split()
            return len(words)
    except FileNotFoundError:
        return 0

if __name__ == "__main__":
    # Read filename from stdin/input
    input_data = sys.stdin.read().split()
    if input_data:
        filename = input_data[0].strip()
        total_words = count_words_in_file(filename)
        print(f"Total words: {total_words}")
```

	Input	Expected	Got	
✓	input1.txt	Total words: 6	Total words: 6	✓
✓	input2.txt	Total words: 14	Total words: 14	✓
✓	input3.txt	Total words: 15	Total words: 15	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a specific line from a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to read.
2. **Output:**
   - The content of the specified line.

input1.txt:

Line one.
Line two.
Line three.
Line four.

For example:

Input	Result
input1.txt 3	Line three.

Answer: (penalty regime: 0 %)

```
import sys

def read_specific_line(filename: str, line_number: int) -> str:
    try:
        with open(filename, 'r', encoding='utf-8') as file:
            for current_line_num, line in enumerate(file, start=1):
                if current_line_num == line_number:
                    # Return line content without trailing newline character
                    return line.rstrip('\r\n')
        return "Line number out of range."
    except FileNotFoundError:
        return f"Error: File '{filename}' not found."

if __name__ == "__main__":
    # Read filename and line number from standard input
    input_data = sys.stdin.read().splitlines()

    if len(input_data) >= 2:
        filename = input_data[0].strip()
        line_num = int(input_data[1].strip())

        result = read_specific_line(filename, line_num)
        print(result)
```

	Input	Expected	Got	
✓	input1.txt 3	Line three.	Line three.	✓
✓	input2.txt 3	Line C.	Line C.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Write a Python program to append a new line at a specific position in a text file, shifting existing lines down.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to insert the new line at.
   - New content for the new line.
2. **Output:**
   - The updated file with the new line inserted at the specified position, shifting the existing lines down in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  3

  Inserted line..

Output:

Line one.
Line two.
Inserted line.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`3`
	`Inserted line.`
		`Line one.`
		`Line two.`
		`Inserted line.`
		`Line three.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
import sys

def insert_line_in_file():
    # Read all inputs from stdin
    raw_inputs = sys.stdin.read().strip().splitlines()

    if len(raw_inputs) < 3:
        # Fallback default variables if testing directly in environment
        input_filename = 'input1.txt'
        line_num = 3
        new_content = "Inserted line."
    else:
        input_filename = raw_inputs[0].strip()
        line_num = int(raw_inputs[1].strip())
        new_content = raw_inputs[2]

    try:
        # Read lines from the input file
        with open(input_filename, 'r', encoding='utf-8') as infile:
            lines = infile.readlines()

        # Ensure every existing line ends with a newline character
        lines = [line if line.endswith('\n') else line + '\n' for line in lines]

        # Format the new content with a trailing newline
        if not new_content.endswith('\n'):
            new_content += '\n'
```

```
28
29          target_index = line_num - 1
30
31          # Insert at specified 0-based position
32          if 0 <= target_index <= len(lines):
33              lines.insert(target_index, new_content)
34          else:
35              lines.append(new_content)
36
37          # Write to output.txt
38          with open('output.txt', 'w', encoding='utf-8') as outfile:
39              outfile.writelines(lines)
40
41      except FileNotFoundError:
42          pass
43
44  if __name__ == "__main__":
45      insert_line_in_file()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		3
		Inserted line.
			Line one.
			Line two.
			Inserted line.
			Line three.
			Line four.
				Line one.
				Line two.
				Inserted line.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		4
		Inserted line D.
			Line A.
			Line B.
			Line C.
			Inserted line D.
				Line A.
				Line B.
				Line C.
				Inserted line D.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Develop a Python program to copy the contents of one file to another file.

Description:

1. **Input:**
   - Source file and destination file names.
2. **Output:**
   - The content of the source file copied to the destination file.

For example:

Test	Input	Result

`with open('output1.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt
	output1.txt
		This is the source file.
		It contains multiple lines of text.
		Here is another line.

Answer: (penalty regime: 0 %)

```
import sys

def copy_file_content(source_filename: str, dest_filename: str) -> None:
    try:
        # Read complete content from source file
        with open(source_filename, 'r', encoding='utf-8') as src:
            content = src.read()

        # Write content to destination file
        with open(dest_filename, 'w', encoding='utf-8') as dest:
            dest.write(content)

    except FileNotFoundError:
        print(f"Error: Source file '{source_filename}' not found.")

if __name__ == "__main__":
    # Read source and destination filenames from stdin
    input_data = sys.stdin.read().strip().splitlines()

    if len(input_data) >= 2:
        source_file = input_data[0].strip()
        dest_file = input_data[1].strip()

        copy_file_content(source_file, dest_file)
```

	Test	Input	Expected	Got	

✓	`with open('output1.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		output1.txt
			This is the source file.
			It contains multiple lines of text.
			Here is another line.
				This is the source file.
				It contains multiple lines of text.
				Here is another line.
					✓

✓	`with open('output2.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		output2.txt
			Hello, world!
			Python programming is amazing.
			Let's copy this text to another file.
				Hello, world!
				Python programming is amazing.
				Let's copy this text to another file.
					✓

✓	`with open('output3.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input3.txt
		output3.txt
			Single line.	Single line.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.